Broken Authentication

Identity, login and session-management weaknesses — Week 6 VAPT Notes

Technical Learning Notes | Prepared from authorized local lab work and OWASP guidance

1. Definition and terminology

Broken Authentication is a broad, commonly used term for weaknesses in verifying a user's identity or maintaining a secure authenticated session. Examples include weak or reused passwords, missing protections against repeated login attempts, insecure password recovery, poor multi-factor authentication implementation, and session identifiers that are predictable or inadequately protected.

OWASP Top 10 category names have changed between editions. In the 2021 edition, many authentication and session-management risks are discussed under A07:2021 Identification and Authentication Failures. Use the category and edition required by the internship instructions, and avoid presenting “Broken Authentication” as the exact title of every OWASP edition.

2. Common weakness patterns

- Weak or common passwords, credential stuffing, and password reuse.
- No rate limiting or suitable abuse detection for repeated login attempts.
- Insecure account recovery or password-reset flows.
- Missing or poorly enforced multi-factor authentication for high-risk access.
- Session IDs that do not rotate after login or privilege changes.
- Sessions that remain valid after logout, password reset, or account disablement.
- Cookies missing suitable Secure, HttpOnly, or SameSite attributes.
- Authentication and authorization confusion—for example, assuming a hidden UI element prevents direct access to a protected server endpoint.

These examples describe possible failure modes. Each requires testing within a permitted scope; do not assume a system has a weakness just because a particular control is not visible in the interface.

3. Illustrative example

A test application accepts unlimited login attempts and uses a weak password policy. An attacker could attempt commonly used or previously leaked credentials at scale. Another example is a session identifier that stays unchanged when a user logs in, which can increase session-fixation risk if an attacker can cause a victim to use a known session.

Safer behavior includes layered rate limits and abuse monitoring, strong password storage, secure recovery workflows, session rotation after authentication and privilege changes, and server-side invalidation when a session should end. Tests must avoid real user accounts and credential attacks unless explicitly authorized; use dedicated test accounts and controlled test cases.

4. Impact and risk

Authentication failures can lead to account takeover, unauthorized access to private data, fraudulent transactions, privilege misuse, and loss of auditability. Impact depends on the affected accounts, privileges, exposed functions, and available safeguards. A low-privilege test account compromise and an administrator account compromise do not have the same impact, so findings should describe the tested role and evidence.

5. Relation to my Week 6 DVWA work

The recorded Week 6 practical evidence focuses on SQL Injection, Reflected XSS, Stored XSS, Burp Suite request/response capture, and a partial OWASP ZAP scan. The current evidence summary does not establish a completed, dedicated Broken Authentication test. Therefore, this note is theoretical background and a proposed future test area—not a claim that a Broken Authentication vulnerability was confirmed in DVWA.

If you extend the lab, use a dedicated test account and document only checks you actually perform. Possible safe checks include whether logout invalidates the session, whether the session identifier rotates after login, whether cookie attributes are set appropriately, and whether repeated failed logins trigger suitable controls. Capture the observed behavior and avoid testing against accounts you do not own.

6. Prevention and secure design

1. Store passwords using a modern, purpose-built password hashing algorithm with an appropriate work factor and unique salts; never store plaintext passwords.
2. Encourage long, unique passwords and screen against known-compromised passwords where feasible. Apply risk-appropriate multi-factor authentication.
3. Rate-limit and monitor authentication attempts while avoiding controls that create easy denial-of-service conditions.
4. Use secure, single-use, time-limited password-reset tokens and prevent account-enumeration leaks in recovery responses.
5. Rotate session identifiers after login and privilege changes; expire sessions appropriately and invalidate them on logout or sensitive account events.
6. Set cookies with appropriate Secure, HttpOnly, and SameSite attributes and require HTTPS throughout authenticated sessions.
7. Enforce authorization on the server for every protected operation; authentication alone does not grant every permission.
8. Log security-relevant events without recording passwords, session tokens, or unnecessary sensitive data.

7. Verification and retesting

Use controlled test accounts. Verify login, logout, session rotation, expiry, password reset, rate limiting, and multi-factor behavior according to the application's requirements. Confirm server-side access controls independently of what the UI displays. Record evidence, expected behavior, actual behavior, and retest status. Mark untested items as “not assessed” rather than “passed”.

8. Key takeaway

Secure authentication requires more than a login form: it includes credential handling, recovery, session lifecycle, abuse resistance, and server-side access checks. In this Week 6 project, Broken Authentication is covered in the notes, but no dedicated finding should be claimed unless additional authorized tests provide evidence.

References for further reading

- OWASP Authentication Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Session Management Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP Top 10:2021 A07 Identification and Authentication Failures — https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/